

Projet 1 : PHP-MYSQL - Organisation du code

1. Le projet

Stagiaire au sein de l'hôtel Chambord, le service comptable vous demande de lui fournir une interface WEB permettant de gérer les salariés de l'entreprise. Un premier travail avait été réalisé par un précédent stagiaire, vous allez reprendre ce travail, organiser le code, proposer une interface d'administration permettant d'ajouter et de supprimer un salarié et de mettre à jour les données.

2. prérequis

1. Télécharger l'archive chambord-salaries.zip et l'extraire dans votre répertoire **public_html**. Cette archive contient plusieurs fichiers notamment le script **listeSalaries.php** et **salaries.sql**.
2. Avec phpMyAdmin, exécuter le script **salaries.sql** dans votre base de données **chambord**. Vous pouvez ensuite tester le script **listeSalaries.php** en modifiant éventuellement les données de connexion (login et mot de passe).

À partir de la table salaries, on obtient la vue suivante :

Connexion réussie

id	nom	prenom	date-naissance	date-embauche	salaire	service
1	Cuset	Jean	1966-06-02	1995-05-15	2400	cuisine
2	Dian	Charles	1972-08-30	2007-04-01	1800	cuisine
3	Paco	Alain	1958-04-25	2002-06-06	2100	commercial
4	Perez	Amanda	1972-01-11	1999-03-11	1700	comptable
5	Jeanu	Thierry	1970-07-11	1998-01-01	1600	comptable
6	Taque	Stephane	1965-04-08	2008-02-01	2000	entretien
7	Devos	Yahn	1965-07-08	2009-02-01	2400	entretien
8	Tettu	Sylvain	1975-07-11	2006-02-01	2000	comptable
9	Triet	Jacques	1968-11-08	2005-10-01	3000	cuisine
10	Jolivet	Olivier	1973-02-23	2000-11-06	1800	commercial
11	Jollet	Marc	1965-04-28	1999-02-01	4000	cuisine
12	Milan	Adrien	1980-04-08	2008-02-01	2000	entretien
13	Cerceau	Gilles	1975-03-18	2008-02-01	2000	entretien
14	Tuche	Yves	1965-04-08	2006-02-01	2000	commercial
15	Trichet	Antoine	1978-11-08	2006-02-01	1600	cuisine
16	Alan	Steven	1976-01-21	2006-02-01	2800	cuisine
17	Dilou	Tristan	1978-01-18	2006-02-01	2500	entretien
18	Jaspe	Anouk	1980-04-08	2008-02-01	2000	entretien
19	Anvers	Tonio	1967-06-06	2008-02-01	2400	entretien
20	Clouzot	Edouard	1955-04-08	1998-02-01	2500	entretien

Copyright © Mon application 2026

À partir de cet exemple, nous allons créer un fichier **header.html** et **footer.html** :

header.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bootstrap demo</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
T3c6CoIi6uLrA9TneEoa7RxnatzjcDSCmG1MXxSR1GAsXEV/Dwwykc2MPK8M2HN"
crossorigin="anonymous">
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
      <div class="container">
        <a class="navbar-brand" href="#">Navbar</a>
```

```

<button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-
bs-target="#navbarSupportedContent" aria-controls="navbarSupportedContent" aria-
expanded="false" aria-label="Toggle navigation">
  <span class="navbar-toggler-icon"></span>
</button>
<div class="collapse navbar-collapse" id="navbarSupportedContent">
  <ul class="navbar-nav me-auto mb-2 mb-lg-0">
    <li class="nav-item">
      <a class="nav-link active" aria-current="page" href="#">Home</a>
    </li>
    <li class="nav-item">
      <a class="nav-link" href="#">Link</a>
    </li>
    <li class="nav-item dropdown">
      <a class="nav-link dropdown-toggle" href="#" id="navbarDropdown"
role="button" data-bs-toggle="dropdown" aria-expanded="false">
        Dropdown
      </a>
      <ul class="dropdown-menu" aria-labelledby="navbarDropdown">
        <li><a class="dropdown-item" href="#">Action</a></li>
        <li><a class="dropdown-item" href="#">Another action</a></li>
        <li><hr class="dropdown-divider"></li>
        <li><a class="dropdown-item" href="#">Something else here</a></li>
      </ul>
    </li>
    <li class="nav-item">
      <a class="nav-link disabled">Disabled</a>
    </li>
  </ul>
  <form class="d-flex" role="search">
    <input class="form-control me-2" type="search" placeholder="Search" aria-
label="Search">
    <button class="btn btn-outline-success" type="submit">Search</button>
  </form>
</div>
</div>
</nav>

```

footer.html

```

<div class="container my-5">
  <p>Copyright &copy; Mon application 2026</p>
</div>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"
integrity="sha384-C6RzsynM9kWDrmNeT87bh950GNyZPhcTNXj1NW7RuBCsyN/o0jlpcV8Qyq46cDfL"
crossorigin="anonymous"></script>
  <script src="main.js"></script>
</body>
</html>

```

Ensuite la vue peut être obtenue avec le code suivant :

listeSalaries.php

```
<?php
    require_once('header.html');
?>

<?php
    require_once('footer.html');
?>
```

Ce code fonctionne mais il est difficile à maintenir et il peut vite devenir incompréhensible, si on souhaite par exemple afficher le nombre de salariés, le salaire minimum, maximum etc.. Il faut donc essayer de le réorganiser.

3. Organisation du code

Dans un premier temps, la vue n'a pas à gérer l'accès à la base de données. Si votre site comporte plusieurs vues, vous allez devoir intervenir dans chaque vue en cas de modification des paramètres de connexion. Il est préférable de créer un fichier contenant les paramètres de connexion que l'on va inclure avec un `require_once` : `require_once('connexion.php')`

connexion.php

```
<?php
    $servername = 'localhost';
    $username = 'sio';
    $password = 'sio';
    $dbname = "chambord";

    //On essaie de se connecter
    try{
        $conn = new PDO("mysql:host=$servername;dbname=$dbname", $username,
        $password);
        //On définit le mode d'erreur de PDO sur Exception
        $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        global $conn ; ①
    }

    /*On capture les exceptions si une exception est lancée et on affiche
    *les informations relatives à celle-ci*/
    catch(PDOException $e){
        echo "Erreur : " . $e->getMessage();
    }
?>
```

① Une variable n'est utilisable que dans le « bloc » où elle est définie et ne peut pas être utilisée

ailleurs. Par « bloc » on entend : le fichier, une fonction, une classe. Pour pouvoir utiliser la variable **\$conn** dans un autre fichier, on utilise le mot clé **global \$conn** et pour l'appeler depuis un autre fichier, on fait de même.

La vue contient le code xHTML. C'est la seule partie qui doit en contenir. Les requêtes SQL doivent être effectuées ailleurs, la vue doit se contenter d'afficher les résultats. Il faut donc créer un nouveau fichier qui va contenir toutes les fonctions qui se chargeront des requêtes SQL.

Par exemple, si on souhaite afficher le nombre de salariés, on va définir une fonction que le nommera `getNbSalaries` qui retournera le résultat de la requête suivante :

fonctions.php

```
<?php

require_once("connexion.php");

function getLesSalaries(){
    try {
        /*Sélectionne toutes les valeurs dans la table contact*/
        global $conn ;
        $sql = "SELECT * FROM salaries";
        $stmt = $conn->prepare($sql);
        $stmt->execute();

        /*Retourne un tableau associatif pour chaque entrée de notre table
        *avec le nom des colonnes sélectionnées en clefs*/
        $resultat = $stmt->fetchAll(PDO::FETCH_ASSOC);
        return $resultat ;

    }

    catch(PDOException $e){
        echo "Erreur : " . $e->getMessage();
    }
}

?>
```

le fichier `listeSalaries.php` se présentera ainsi :

```
<?php
require_once('header.html');
require_once("fonctions.php");

$lesSalaries = getLesSalaries();
?>

<div class="container my-5">
```

```

<table class="table table-hover">
  <th>id</th>
  <th>nom</th>
  <th>prenom</th>
  <th>date-naissance</th>
  <th>date-embauche</th>
  <th>salaire</th>
  <th>service</th>
  <?php foreach ($lesSalaries as $leSalarie): ?>
    <tr>
      <td><?= htmlspecialchars( $leSalarie['id']); ?></td>
      <td><?= htmlspecialchars( $leSalarie['nom']); ?></td>
      <td><?= htmlspecialchars($leSalarie['prenom']); ?></td>
      <td><?= htmlspecialchars( $leSalarie['date_naissance']); ?></td>
      <td><?= htmlspecialchars( $leSalarie['date_embauche']); ?></td>
      <td><?= htmlspecialchars( $leSalarie['salaire']); ?></td>
      <td><?= htmlspecialchars( $leSalarie['service']); ?></td>
    </tr>
  <?php endforeach; ?>
</table>
</div>

<?php
require_once('footer.html');
?>

```

3.1. Redirection et pattern PRG

3.1.1. Redirection

Il existe des applications pour lesquelles on souhaite rediriger le visiteur en fonction de paramètres. C'est le cas par exemple pour un script d'identification. Si l'internaute fournit les bons identifiants alors il est redirigé automatiquement vers son espace personnel, sinon il est renvoyé vers le formulaire d'authentification. Lorsqu'un utilisateur remplit un formulaire, on récupère les données pour les insérer dans une base et ensuite on redirige l'utilisateur vers une autre page.

PHP dispose de la fonction `header()` qui se charge d'envoyer les entêtes passés en paramètre.



l'appel la fonction `header()` doit se faire avant tout envoi au navigateur (instruction `echo`, `print`, espace blanc, balise `html`...) sous peine de générer une erreur de type `Headers already sent by...` Cette erreur signifie que la page a déjà été envoyée au navigateur avant de vouloir envoyer des entêtes HTTP. La logique de développement demande le contraire !

```

<?php
header('Location: http://www.votresite.com/pageprotegee.php');
exit();

```

Note : l'instruction `exit()` qui succède la fonction `header()` permet de couper l'exécution du script car la redirection aura lieu immédiatement et le reste du code n'a pas d'intérêt à être interprété.

3.1.2. Pattern PRG : Post/Redirect/Get

En général, un formulaire HTML envoie des données au serveur via la méthode POST. Le script serveur récupère ces données pour les traiter, par exemple en ajoutant un enregistrement dans une base de données ou en exécutant une requête. Si l'utilisateur actualise accidentellement la page, les mêmes données peuvent être renvoyées, ce qui risque d'entraîner une perte d'intégrité des données. L'approche PRG en PHP permet d'éviter cet écueil.

Pour éviter cette situation, il est préférable que les données soient traitées sur une page PHP qui n'affichera rien (éventuellement une confirmation JavaScript) et qui redigera l'utilisateur sur une autre page. (pour une sécurité plus forte il faut créer des sessions).

Pour expliquer le fonctionnement du modèle Post/Redirect/Get, nous prendrons l'exemple d'une commande sur un site de commerce électronique. Le processus se divise en trois étapes :

1. **Étape 1 (POST)** : L'utilisateur finalise sa commande et envoie une requête POST au serveur via son navigateur. Le serveur traite alors cette requête et enregistre la commande dans une base de données.
2. **Étape 2 (REDIRECTION)** : Au lieu d'envoyer directement la page de confirmation à l'utilisateur, le serveur répond au navigateur par une redirection (avec le code d'état 3xx) vers une nouvelle URL, qui conduit à la page de confirmation.
3. **Étape 3 (GET)** : En raison de la redirection, le navigateur envoie alors une nouvelle requête GET au serveur (puisque'il doit récupérer une nouvelle URL), afin de demander la page de confirmation. Si l'utilisateur actualise la page, le navigateur envoie une nouvelle requête GET et le serveur répond par la page de confirmation.

Sans le modèle **PRG**, l'utilisateur renverrait la requête POST en actualisant la page, et le serveur enregistrerait la commande deux fois.

4. travail à faire :

4.1. Première partie (à faire seul durée 4h)

1. Modifier le script `listeSalaries.php` pour séparer la vue du traitement des données. Vous allez devoir créer un fichier `connexion.php` et un fichier `fonctions.php` qui contiendra toutes les fonctions utiles à la vue.
2. Écrire les fonctions qui vont vous permettre d'ajouter à la vue :
 1. le nombre de salariés

2. le salaire moyen
3. le salaire maximum et minimum
4. le nombre de salariés par service

5. Deuxième partie : interface d'administration (8h)

Vous pouvez éventuellement travailler en groupe, mais vous devez rendre un rapport individuel.

1. en vous aidant de l'activité 1, proposez une vue permettant de réaliser toutes les opérations CRUD (ajout d'un salarié, suppression et mise à jour)

Navbar Home Link Dropdown Disabled

Search

Search

+ Ajouter salarié

id	nom	prenom	date-naissance	date-embauche	salaire	service	update	delete
1	Cuset	Jean	1966-06-02	1995-05-15	2400	cuisine	 Update	 Supprimer
2	Dian	Charles	1972-08-30	2007-04-01	1800	cuisine	 Update	 Supprimer
3	Paco	Alain	1958-04-25	2002-06-06	2100	commercial	 Update	 Supprimer
4	Perez	Amanda	1972-01-11	1999-03-11	1700	comptable	 Update	 Supprimer
5	Jeanu	Thierry	1970-07-11	1998-01-01	1600	comptable	 Update	 Supprimer
6	Taque	Stephane	1965-04-08	2008-02-01	2000	entretien	 Update	 Supprimer
7	Devos	Yahn	1965-07-08	2009-02-01	2400	entretien	 Update	 Supprimer
8	Tettu	Sylvain	1975-07-11	2006-02-01	2000	comptable	 Update	 Supprimer
9	Triet	Jacques	1968-11-08	2005-10-01	3000	cuisine	 Update	 Supprimer

1. Créer sur votre **GIT** un nouveau **repository** chambord-salaries et pousser votre code final dessus.
2. Proposer une ou des solutions pour protéger les pages permettant de gérer les salariés.